

Currency Exchange Sprint 1

Secrets, Configuration, File-by-File Explanation, and Startup Guide

This PDF combines the detailed explanation of secret templating with a practical guide to each project file and the startup flow.

Currency Exchange Sprint 1

Secrets, Configuration, File-by-File Explanation, and Startup Guide

This document combines two needs:

1. A detailed explanation of how secrets are templated and configured.
2. A file-by-file explanation of the project scaffold, with practical startup and operating steps.

The goal is to help junior developers understand not only **what** to change, but also **why** each file exists, **how** the values connect to each other, and **what happens** when the stack starts.

1. Project purpose and mental model

This project is a learning environment for a fictional currency exchange business. The goal of Sprint 1 is not to build the final commercial product. The goal is to create a small but realistic platform that teaches how the major pieces of a modern web application fit together:

- **Django** handles the web application and server-rendered pages.
- **Keycloak** handles login, registration, logout, and account identity.
- **PostgreSQL** stores application and identity data.
- **Nginx** acts as the reverse proxy that the browser talks to first.
- **Docker Compose** runs all of the services together in a repeatable way.

The project is intentionally split into separate containers because that makes each responsibility easier to understand and debug.

A junior developer should think about the stack like this:

- the browser sends requests to Nginx,
- Nginx forwards them to Django or Keycloak,
- Django renders the user-facing pages,
- Keycloak manages identity,
- PostgreSQL persists data,
- Docker keeps the environment reproducible.

That separation is a very common pattern in real-world systems.

2. Why secrets must be templated

A secret is any value that should not be hardcoded into the source code and should not be shared publicly. In this project, the most important secret values are:

- Django secret key
- PostgreSQL passwords
- Keycloak admin password
- Keycloak client secret

Secrets are templated so the same codebase can be reused by different developers and environments without rewriting the application code.

Instead of writing secrets directly into Python files or JSON files, the project uses environment variables. That means:

- the code reads values at runtime,
- the values can change without changing the code,
- each developer can have their own local ``.env`` file,
- the repository can include a safe ``.env.example`` file for guidance.

This is a much safer and cleaner pattern than hardcoding credentials.

3. The main configuration files and what each one does

3.1 ``.env.example``

This is the template file. It shows the names of the variables the project expects.

Why it exists

A new developer can copy this file to ``.env`` and fill in the values. It acts like a checklist.

What should be inside

- variable names
- safe example values
- short comments describing the variables

*What should ****not**** be inside*

- real passwords
- real production secrets
- private API keys

Example idea

```
DJANGO_SECRET_KEY=change-me  
POSTGRES_PASSWORD=change-me  
KEYCLOAK_ADMIN_PASSWORD=change-me
```

These are placeholders. They tell the developer exactly what must be replaced.

3.2 ``.env``

This is the real runtime file used by Docker Compose and Django.

Why it exists

Docker Compose automatically reads ``.env`` when it starts. That makes it easy to inject real values into containers.

What should be inside

- the actual secret values for that machine
- the chosen realm/client names
- the local URLs and ports

What should happen to it

- keep it local
- do not commit it in a real project
- do not share it casually

Why this matters

If someone changes the file, the application should still work because the code reads the environment variables instead of hardcoded values.

3.3 ``docker-compose.yml``

This file defines the services in the stack.

Why it exists

It lets the team start all services with one command rather than starting each container manually.

Services typically defined

- ``db`` for Django's PostgreSQL database
- ``keycloak-db`` for Keycloak's PostgreSQL database
- ``keycloak`` for the identity provider
- ``web`` for Django
- ``nginx`` for reverse proxy routing

Why it uses environment variables

The Compose file should not store secrets directly. Instead, it uses values like:

```
POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
KEYCLOAK_ADMIN_PASSWORD: ${KEYCLOAK_ADMIN_PASSWORD}
```

This means Docker Compose substitutes the values from ``.env`` at runtime.

Why that is useful

- the Compose file remains readable
- secrets stay outside the code
- the same Compose file can be reused in different environments

3.4 ``backend/Dockerfile``

This file builds the Django container image.

Why it exists

Docker needs a recipe for building the application image. The Dockerfile is that recipe.

What it does

- starts from a Python base image
- installs system packages needed for Python and PostgreSQL
- installs the Python requirements
- copies the Django code into the container
- starts Gunicorn

Why it should not contain secrets

The image should be reusable. If secrets were baked into the image, every rebuild would risk exposing them and the image would be tied to one environment.

What to remember

The Dockerfile defines the `**environment**`, not the `**secret values**`.

3.5 `backend/requirements.txt`

This lists the Python packages the app needs.

Why it exists

Python dependencies should be recorded in one place so every developer installs the same versions.

Typical packages here

- Django
- Gunicorn
- psycopg2-binary
- python-dotenv
- mozilla-django-oidc

Why this matters

If every developer uses slightly different package versions, the app can behave differently on different machines.

3.6 `backend/config/settings.py`

This is the main Django settings file.

Why it exists

Django reads this file to know how to configure itself.

What it should read from the environment

- `DJANGO\_SECRET\_KEY`
- `DJANGO\_DEBUG`
- `DJANGO\_ALLOWED\_HOSTS`

- `POSTGRES\_DB`
- `POSTGRES\_USER`
- `POSTGRES\_PASSWORD`
- `POSTGRES\_HOST`
- `POSTGRES\_PORT`
- `KEYCLOAK\_SERVER\_URL`
- `KEYCLOAK\_REALM`
- `KEYCLOAK\_CLIENT\_ID`
- `KEYCLOAK\_CLIENT\_SECRET`
- redirect and site URLs

Why each value matters

- **secret key**: signs cookies and protects Django internals
- **debug**: enables development mode; must not stay on in production
- **allowed hosts**: prevents host header attacks and config errors
- **database settings**: let Django connect to PostgreSQL
- **Keycloak settings**: tell Django where login and token endpoints are
- **redirect URLs**: ensure users return to the right page after login or logout

Why this file is important

This file is the central bridge between the code and the environment.

3.7 `backend/config/urls.py`

This file maps URL paths to views.

Why it exists

When the browser requests `/dashboard/`, Django needs to know what code should respond.

What it connects

- home page
- dashboard
- account page
- login page
- logout page
- register page
- delete account page
- OIDC routes for Keycloak integration

Why it matters for learning

This is where juniors start seeing how routes become real pages.

3.8 `backend/core/views.py`

This file contains the application logic for the pages.

Why it exists

Views are the code that decides what the browser sees.

Current responsibilities

- show the landing page
- show the dashboard
- show the account page
- redirect the login flow to OIDC
- redirect the registration flow to Keycloak registration
- log the user out
- show the delete account confirmation page

Why it is kept simple in Sprint 1

The aim is to prove the login and identity flow first. More advanced business logic can come later.

3.9 `backend/core/templates/core/\*.html`

These are the HTML templates for the user interface.

Why they exist

Django templates let the server render pages without requiring a front-end framework.

Files included

- `base.html`
- `home.html`
- `dashboard.html`
- `account.html`
- `delete\_account.html`

What each one does

- **base.html**: shared layout and styling
- **home.html**: public landing page
- **dashboard.html**: logged-in landing area
- **account.html**: account details and delete option
- **delete_account.html**: confirmation page for deletion flow

Why Django templates are a good choice here

They are easier to understand for juniors than a separate SPA architecture.

3.10 `nginx/default.conf`

This file configures the reverse proxy.

Why it exists

The browser should talk to Nginx, not directly to every container.

What it routes

- general requests to Django
- OIDC-related paths to Django
- Keycloak paths to Keycloak

Why it matters

It hides the internal container layout from the user and makes the browser experience cleaner.

3.11 `keycloak/realm-export.json`

This file seeds the Keycloak realm.

Why it exists

It gives the team a repeatable identity configuration.

What it can contain

- realm name
- client definition
- redirect URIs
- role definitions
- example users
- registration settings

Why it is useful

Instead of creating everything manually from scratch every time, the project can start from a known baseline.

Important note

Use safe placeholders for learning. Do not store real secrets in this file.

3.12 `postgres/init/01-create-databases.sql`

This file is an example SQL bootstrap script.

Why it exists

It shows how a database and user might be created manually.

What it teaches

- how databases are created
- how users are granted privileges
- how identity data and application data can be separated

Why it may be a reference rather than active config

The Compose setup may already handle the databases through container environment variables. Even so, the SQL file is a useful learning artifact.

3.13 `README.md`

This file explains how to start the project and what the repository contains.

Why it exists

It helps a developer start without having to guess the structure.

3.14 `.gitignore`

This file tells Git what to ignore.

Why it exists

It keeps local secrets and generated files out of the repository.

Typical exclusions

- `.env`
- Python bytecode
- caches
- local editor files
- build artifacts

3.15 `backend/.dockerignore`

This file tells Docker what not to send into the image build.

Why it exists

It makes Docker builds faster and avoids copying unnecessary local files into the image.

Typical exclusions

- `.env`
- `.git`
- caches
- generated files

4. Which secrets need to be configured and why

These values should be reviewed and changed from the placeholders before serious use.

4.1 `DJANGO_SECRET_KEY`

Why it matters

Django uses it for cryptographic signing of sessions and cookies.

What happens if it is weak or exposed

- session security is weakened
- the app becomes easier to tamper with

Recommendation

Use a long random value.

4.2 `POSTGRES\_PASSWORD`

Why it matters

This protects the PostgreSQL user for the Django database.

What happens if it is weak

Anyone with network access to the database container could potentially authenticate more easily.

Recommendation

Use a strong unique password.

4.3 `KEYCLOAK\_DB\_PASSWORD`

Why it matters

This protects the database behind Keycloak.

Why Keycloak depends on it

Keycloak stores users, clients, realms, and identity-related data there.

Recommendation

Use a unique password different from the Django database password.

4.4 `KEYCLOAK\_ADMIN\_PASSWORD`

Why it matters

This is the password for the Keycloak admin user.

What happens if it leaks

Someone could administer the whole realm and its authentication settings.

Recommendation

Use a strong password and treat it as highly sensitive.

4.5 `KEYCLOAK\_CLIENT\_SECRET`

Why it matters

Django uses this to authenticate itself as a confidential OIDC client.

What happens if it leaks

The trust relationship between Django and Keycloak can be compromised.

Recommendation

Keep it private and synchronized between Keycloak and Django.

4.6 Non-secret values that still must match

These values are not secrets, but they must be correct:

- ``KEYCLOAK_REALM``
- ``KEYCLOAK_CLIENT_ID``
- ``DJANGO_ALLOWED_HOSTS``
- ``SITE_URL``
- ``LOGIN_REDIRECT_URL``
- ``LOGOUT_REDIRECT_URL``

If these do not match the running stack, login flows and browser redirects can fail.

5. Steps to do beforehand

Before starting the stack, do these steps in order.

Step 1: install prerequisites

Make sure the machine has:

- Docker Engine
- Docker Compose plugin
- Git
- VSCode or another code editor

Why this matters

Without these tools, the junior developer will spend time troubleshooting missing commands instead of learning the stack.

Step 2: copy ``.env.example`` to ``.env``

Create a real local environment file.

Why this matters

The example file is only a template. The actual runtime values belong in ``.env``.

Step 3: set the secret values

Update the following in ``.env``:

- ``DJANGO_SECRET_KEY``
- ``POSTGRES_PASSWORD``
- ``KEYCLOAK_DB_PASSWORD``
- ``KEYCLOAK_ADMIN_PASSWORD``
- ``KEYCLOAK_CLIENT_SECRET``

Why this matters

If these values are left as placeholders, the stack will either be insecure or fail to work correctly.

Step 4: confirm the ports are free

The stack uses:

- port 80 for Nginx
- port 8080 for Keycloak
- port 5432 for PostgreSQL if exposed locally

Why this matters

If another process already uses one of these ports, containers may start but be unreachable.

Step 5: review the project folder structure

The main folders are:

- ``backend/``
- ``nginx/``
- ``keycloak/``
- ``postgres/``

Why this matters

Knowing where each file lives makes debugging much easier.

Step 6: make sure secrets are not committed

Check that ``gitignore`` includes ``env``.

Why this matters

This avoids accidental exposure of passwords and secret keys.

6. Step-by-step startup guide

Step 1: open the project folder

Work from:

`C:\Users\mpenayo\Documents\Agile Learning Project\currency-exchange-sprint1`

Step 2: build and start the stack

```
docker compose up -d --build
```

What this does

- builds the Django image
- starts the databases
- starts Keycloak
- starts Django
- starts Nginx

Step 3: confirm the services are running

```
docker compose ps
```

What to look for

- all services in a healthy or running state
- no immediate restarts

Step 4: inspect logs if needed

```
docker compose logs -f web
docker compose logs -f keycloak
docker compose logs -f nginx
```

Why logs matter

Logs show the exact error message rather than guessing why a service failed.

Step 5: open the browser

Visit:

- `http://localhost`
- `http://localhost:8080`

What these mean

- `http://localhost` goes through Nginx to the app
- `http://localhost:8080` opens Keycloak directly

7. How to configure everything once the stack is running

7.1 PostgreSQL

What to check

- database container is healthy
- Django can connect
- migrations run

If something fails

Check the values in `.env`:

- `POSTGRES_DB`
- `POSTGRES_USER`
- `POSTGRES_PASSWORD`
- `POSTGRES_HOST`
- `POSTGRES_PORT`

Why this matters

Django depends on the database before it can load normal app pages.

7.2 Keycloak

Open:

`http://localhost:8080`

Log in using:

- ``KEYCLOAK_ADMIN``
- ``KEYCLOAK_ADMIN_PASSWORD``

Then verify

- realm name is ``exchange-learning``
- client ID is ``exchange-web``
- redirect URIs are correct
- registration is enabled if required for Sprint 1
- roles exist if the JSON imported them

Why this matters

If the realm or client settings do not match Django, login will fail even if Keycloak itself is running.

7.3 Django

What to check

- homepage loads
- login redirects to Keycloak
- logout returns to the landing page
- dashboard is protected

Useful manual migration command

```
docker compose run --rm web python manage.py migrate
```

Why this matters

Django will not work correctly if its database tables are not initialized.

7.4 Nginx

What to check

- the browser can access the app through port 80
- Nginx can reach Django
- Nginx can route auth-related requests correctly

Why this matters

Nginx is the outer door of the application for the browser.

7.5 Frontend pages

Check that these pages work:

- `/`
- `/login/`
- `/register/`
- `/dashboard/`
- `/account/`
- `/delete-account/`

Why this matters

The user journey should be visible from public landing page to authenticated account management.

8. Recommended post-start checklist

After the stack is running, do the following:

1. open the landing page
2. verify the currency exchange branding appears
3. open Keycloak admin console
4. verify the realm imported correctly
5. check the client configuration
6. test login from the browser
7. test logout
8. test the account page after login
9. test the delete-account page
10. review logs for errors
11. confirm that data persists in the Docker volumes

Why this checklist matters

It gives the team a repeatable validation sequence and prevents random testing.

9. Development workflow after startup

A good workflow for juniors is:

1. change one file
2. restart the relevant container if needed
3. inspect logs
4. test one specific flow
5. commit to a feature branch
6. open a pull request

Why this approach works

It keeps debugging simple and helps the team see which change caused which effect.

10. Common mistakes and how to avoid them

Mistake 1: hardcoding secrets into Python files

Avoid this by reading values from the environment.

Mistake 2: committing `.env`

Avoid this by keeping `.env` out of Git.

Mistake 3: changing too many files at once

Avoid this by making small, focused changes.

Mistake 4: forgetting to align Keycloak and Django settings

Avoid this by verifying realm names, client IDs, and redirect URIs.

Mistake 5: ignoring logs

Avoid this by reading the logs before changing code blindly.

11. Short explanation of the architecture, repeated with context

This learning stack is built the way many real systems are built:

- Django is the app layer,
- Keycloak is the identity layer,
- PostgreSQL is the persistence layer,
- Nginx is the traffic entry point,
- Docker Compose ties the services together.

That is why the files are separated the way they are. Each file exists to make one part of the system understandable and maintainable.

12. Final order recommended for the team

If the juniors want the least confusing sequence, they should do the work in this order:

1. prepare `.env`
2. start PostgreSQL
3. start Keycloak
4. start Django
5. add Nginx routing
6. test login/logout
7. improve the frontend

8. then expand the account management behavior

Why this order is best

It follows dependency order and reduces the amount of simultaneous debugging.

13. Final reminder

The most important rule for this sprint is:

****Keep secrets out of the code, keep configuration in environment variables, and keep each file responsible for only one job.****

That single habit will make the project much easier to understand and much safer to maintain.